neoway

Get connected Get smart

OpenCPU SDK

OS API Notes

Issue 1.0
Date 2025-01-23

Notice

This document is specifically for **N706B** and **N520** modules.

This document is intended for system engineers (SEs), development engineers, and test engineers.

THIS DOCUMENT PROVIDES SUPPORT FOR PRODUCT DESIGN BY THE USER. THE USER MUST DESIGN AND DEBUG THE PRODUCT ACCORDING TO THE SPECIFICATIONS AND PARAMETERS IN THIS DOCUMENT. NEOWAY ASSUMES NO RESPONSIBILITY FOR PERSONAL INJURY AND PROPERTY LOSS CAUSED BY USER'S IMPROPER OPERATION.

DUE TO PRODUCT VERSION UPDATES OR OTHER REASONS, THE CONTENT OF THIS DOCUMENT WILL BE UPDATED AS NECESSARY WITHOUT PRIOR NOTICE.

UNLESS OTHERWISE AGREED, ALL STATEMENTS, INFORMATION AND SUGGESTIONS IN THIS DOCUMENT CONSTITUTE NO WARRANTY WHETHER EXPRESS OR IMPLIED.

Neoway Technology Co., Ltd. provides customers with comprehensive technical support. For any inquiry, please contact your account manager directly or send an email to the following email address:

Sales@neoway.com

Support@neoway.com

Website: http://www.neoway.com

Contents

About This Document

Scope

This document is applicable to N706B and N520 modules.

Audience

This document is intended for system engineers (SEs), development engineers, and test engineers.

Change History

Issue	Date	Change	Changed By
1.0	2023-05	Initial release	Hu Jun

Conventions

Symbol	Indication
	This warning symbol means danger. You are in a situation that could cause fatal device damage or even bodily damage.
	Means reader be careful. In this situation, you might perform an action that could result in module or product damages.
	Means note or tips for readers to use the module

1 Overview

This document introduces the APIs related to system operations in OpenCPU SDK, covering the following aspects:

- Device Information

  Used to obtain module details such as model, version, IMEI, and other system resources.

- Message Queue

  Covers the creation of message queues, message sending, and receiving.

- File Operation

  Defines operations related to files and paths, such as opening, closing, reading, and writing.

- Thread Operations

  Used for thread creation, state management, and thread event handling.

- Synchronization Locks

  Covers the creation, locking, unlocking, and deletion of synchronization locks.

- Semaphores

  Used for creating, acquiring, and releasing semaphores.

- Timer

  Covers the creation, starting, stopping, and destruction of timers.

- Date and Time

  Used for retrieving the system time.

- System Logs

  Provides functionality for LOG output.

2 Data Structure

2.1 typedef enum nwy_error_e

2.2 typedef struct nwy_time_t

```
typedef struct nwy_time_t
{
   unsigned short year;  // [0,127] Year-2000, tm_year: Year-1900
   unsigned char mon;  // [1,12], tm_mon: (0-11)
   unsigned char day;   // [1,31], tm_mday: (1-31)
   unsigned char hour;  // [0,23], tm_hour: (0-23)
   unsigned char min;   // [0,59], tm_min: (0-59)
   unsigned char sec;   // [0,59], tm_sec: (0-60)
   unsigned char wday;  // [1,7] 1 for Monday, tm_wday: (0-6) 0 for Sunday
} nwy_time_t;
```

3 APIs

3.1 Device Information

This section introduces the device management APIs.

The API definitions can be found in **nwy_common.h** and **nwy_dm_api.h**, and is used for device-related operations.

3.1.1 nwy_dm_dev_model_get

Function	nwy_error_e nwy_dm_dev_model_get(char *model_buf, int buf_len);
Description	Obtains device information
Parameter	model_buf: device information(e.g.: N706B) buf_len: buffer length
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.1.2 nwy_dm_inner_version_get

Function	nwy_error_e nwy_dm_inner_version_get(char *version_buf, int buf_len);
Description	Queries the firmware version
Parameter	version_buf: underlying version number buf_len: buffer length
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.1.3 nwy_dm_imei_get

Function	nwy_error_e nwy_dm_imei_get(nwy_dev_simid_e simid, char* imei_buf, int buf_len);
Description	Obtains IMEI of the device
Parameter	simid: SIM card id value, defined by reference to nwy_dev_simid_e. imsi_buf: buffer used to return IMEI, ensure that the buffer length is greater than 16 bytes. buf_len: imsi_buf length
Return Value	Success: **NWY_SUCCESS**

	Failure: another value

3.1.4 nwy_dm_hw_version_get

Function	nwy_error_e nwy_dm_hw_version_get(char *version_buf, int buf_len);
Description	Obtains the hardware version number
Parameter	version_buf: buffer used to return IMEI, ensure that the buffer length is greater than 16 bytes. buf_len: version_buf length
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.1.5 nwy_dm_app_version_set

Function	nwy_error_e nwy_dm_app_version_set(char * version_buf, int buf_len);
Description	Sets the customer APP version information. After the version number is set, you can execute AT+NAPPCHECK? to query the result.
Parameter	version_buf: Sets the content of the customer APP version number, with a length not exceeding 128 bytes. buf_len: version_buf length
Return Value	NA

3.1.6 nwy_dm_sn_get

Function	nwy_error_e nwy_dm_sn_get(char *sn, int len);
Description	Obtains the SN number of the product.
Parameter	sn: output the SN number of the product in character string.
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.1.7 nwy_dm_rftemperature_get

Function	nwy_error_e nwy_dm_rftemperature_get(float *outvalue)
Description	Obtains device RF temperature.
Parameter	outvalue: Output the acquired RF temperature value.
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.1.8 nwy_dm_heapinfo

Function	nwy_error_e nwy_dm_heapinfo(nwy_heapinfo_t * heapinfo);
Description	Retrieves the remaining dynamic RAM information.

Parameter	typedef struct { uint32 start; uint32 size; uint32 avail_size; uint32 max_block_size; }nwy_heapinfo_t; stack: stack starting address size: stack data length available_ram: remaining available size of ram max_block_size: Maximum allocatable size of RAM
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.2 Message Queue

Performs the message-queue-related functions. The API definitions can be found in **nwy_osi_api.h**.

3.2.1 nwy_msg_queue_create

Function	nwy_error_e nwy_msg_queue_create (nwy_osi_msg_queue_t **msg_q, uint32 msg_size, uint32 msg_num)
Description	Initializs the message queue
Parameter	msg_num: Maximum number of messages the message queue can hold. msg_size: Size of each message content. msg_q: Pointer to the created message queue, represented as a pointer to a pointer.
Return Value	Success: **NWY_SUCCESS** Failure: another enumerated value

3.2.2 nwy_msg_queue_delete

Function	nwy_error_e nwy_msg_queue_delete (nwy_osi_msg_queue_t *msg_q)
Description	Deletes a message queue
Parameter	msg_q: message queue
Return Value	Success: **NWY_SUCCESS** Failure: another enumerated value
Remarks	After using the message queue, you must delete the message queue; otherwise a memory leak will occur.

3.2.3 nwy_msg_queue_send

Function	nwy_error_e nwy_msg_queue_send(nwy_osi_msg_queue_t *msg_q, uint32 size, void* msg_ptr, int timeout)
Description	Sends message
Parameter	msg_q: message queue size: message length *msg_ptr: message content timeout: Behavior when the msgqueue is full. • 0: no block • 1: wait for ever • Other values: wait for time ms
Return Value	Success: **NWY_SUCCESS** Failure: another enumerated value

3.2.4 nwy_msg_queue_recv

Function	nwy_error_e nwy_msg_queue_recv(nwy_osi_msg_queue_t *msg_q, uint32 size, void* msg_ptr, int timeout)
Description	Receives messages
Parameter	msg_q: message queue size: message length *msg_ptr: message content timeout: Behavior when the msgqueue is empty • 0: no block • 1: wait for ever • Other values: wait for time ms
Return Value	Success: **NWY_SUCCESS** Failure: another enumerated value

3.2.5 nwy_msg_queue_get_pending_num

Function	nwy_error_e nwy_msg_queue_get_pending_num(nwy_osi_msg_queue_t *msg_q, uint32 *pnum);
Description	Obtains the number of messages to be processed in the queue.
Parameter	msg_q: message queue pnum: number of returned messages
Return Value	Success: **NWY_SUCCESS** Failure: another enumerated value

3.2.6 nwy_msg_queue_get_space_num

Function	nwy_error_e nwy_msg_queue_get_space_num(nwy_osi_msg_queue_t *msg_q,

	uint32 *pnum);
Description	Gets the number of available message slots in the queue
Parameter	msg_q: message queue pnum: pointer to the variable where the available space count in the message queue will be stored.
Return Value	Success: **NWY_SUCCESS** Failure: another enumerated value

3.3 File Operation

To perform the message-queue-related functions. The API definitions can be found in **nwy_file.h**.

3.3.1 nwy_file_open

Function	int nwy_file_open(const char *path, nwy_file_openmode_e mode);
Description	Opens a file
Parameter	*path: file path mode: file open methods, refer to nwy_file_open_mode_e
Return Value	Success: file descriptor Failure: value <0

3.3.2 nwy_file_close

Function	nwy_error_e nwy_file_close(int fd);
Description	Closes a file
Parameter	fd: file descriptor
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.3.3 nwy_file_read

Function	int nwy_file_read(int fd, void *dst, int size);
Description	Read a file
Parameter	fd: File descriptor of the file to be read. *dst: Pointer to the buffer where the read data will be stored. size: Number of bytes to read.
Return Value	>0: Number of bytes successfully read. = 0: End of file reached. < 0: An error occurred.

3.3.4 nwy_file_write

Function	int nwy_file_write(int fd, const void *data, int size);
Description	Write data to a file
Parameter	fd: file descriptor *length: data to be written size: data bytes
Return Value	Success: bytes of written data Failure: another value

3.3.5 nwy_file_seek

Function	int nwy_file_seek(int fd, int offset, nwy_fseek_offset_e mode)
Description	Set a file pointer
Parameter	fd: file descriptor offset: offset mode: where to start the offset, see nwy_fseek_offset_e
Return Value	Success: offset Failure: another value

3.3.6 nwy_file_path_size

Function	long nwy_file_path_size(const char *path);
Description	Obtain the file size.
Parameter	path: file name
Return Value	Success: file size in bytes Failure: -1

3.3.7 nwy_file_fd_size

Function	long nwy_file_fd_size(int fd);
Description	Obtain the file size.
Parameter	fd: File descriptor of the file to be obtained.
Return Value	Success: file size in bytes Failure: -1

3.3.8 nwy_file_exist

Function	bool nwy_file_exist(const char *path);
Description	Check if the file exists
Parameter	path: file path

Return Value	True: existence False: no existence

3.3.9 nwy_file_remove

Function	nwy_error_e nwy_file_remove(const char* path);
Description	Delete a file
Parameter	path: file path
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.3.10 nwy_file_rename

Function	nwy_error_e nwy_file_rename(const char *oldpath, const char *newpath);
Description	Rename a file
Parameter	oldpath: the old file name newpath: the new file name
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.3.11 nwy_file_sync

Function	nwy_error_e nwy_file_sync(int fd);
Description	Sync files
Parameter	fd: file descriptor
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.3.12 nwy_file_fd_trunc

Function	int nwy_file_fd_trunc(int fd, long len);
Description	File interception
Parameter	fd: file descriptor len: file interception length
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.3.13 nwy_file_path_trunc

Function	int nwy_file_path_trunc(const char *path, long len);
Description	File interception

Parameter	path: file name len: file interception length
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.3.14 nwy_file_path_stat_get

Function	nwy_error_e nwy_file_path_stat_get(const char *path, nwy_file_stat *st);
Description	Obtain the file status
Parameter	path: file name st: file information, see stat.h
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.3.15 nwy_file_fd_stat_get

Function	nwy_error_e nwy_file_fd_stat_get(int fd, nwy_file_stat *st);
Description	Obtain the file status
Parameter	fd: file descriptor st: file information, see stat.h
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.3.16 nwy_dir_mk

Function	nwy_error_e nwy_dir_mk(const char *path);
Description	Create a directory
Parameter	path: directory path
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.3.17 nwy_dir_rm

Function	nwy_error_e nwy_dir _rm(const char *path);
Description	Delete a directory
Parameter	path: directory path
Return Value	Success: **NWY_SUCCESS** Failure: another value
Remarks	Non-empty folders cannot be deleted.

3.3.18 nwy_dir_recursive_rm

Function	nwy_error_e nwy_dir_recursive_rm(const char *path);
Description	Delete a directory
Parameter	path: directory path
Return Value	Success: **NWY_SUCCESS** Failure: another value
Remarks	Forcibly delete a directory, including non-empty folder

3.3.19 nwy_dir_open

Function	nwy_dir_info_t *nwy_dir_open(const char *name);
Description	Open a file directory.
Parameter	name: directory name
Return Value	nwy_dir_info_t: Represents an opened directory pointer. Returns NULL if the operation fails.

3.3.20 nwy_dir_close

Function	nwy_error_e nwy_dir_close(nwy_dir_info_t *pdir);
Description	Close the opened directory
Parameter	pdir: Directory address to be closed.
Return Value	N/A.

3.3.21 nwy_dir_read

Function	nwy_error_e nwy_dir_read(nwy_dir_info_t *pdir, nwy_dirent_t *d_info)
Description	Read directory content
Parameter	pdir: Directory address to be read
Return Value	nwy_dirent: directory information

3.3.22 nwy_dir_tell

Function	int nwy_dir_tell(nwy_dir_info_t *pdir);
Description	Get the current read position of the directory stream.
Parameter	pdir: directory address d_info: read information
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.3.23 nwy_dir_seek

Function	nwy_error_e nwy_dir_seek(nwy_dir_info_t *pdir, long loc);
Description	Set the current read position of the directory stream specified by the parameter pdir. After setting the position, subsequent calls to readdir() will begin reading from the specified new position.
Parameter	pdir: directory address loc: The offset from the beginning of the directory file, indicating the new position for reading.
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.3.24 nwy_dir_rewind

Function	nwy_error_e nwy_dir_rewind(nwy_dir_info_t *pdir);
Description	Move the location pointer to the beginning of the file
Parameter	pdir: directory address
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.3.25 nwy_vfs_free_size_get

Function	nwy_error_e nwy_vfs_free_size_get(unsigned long long *avail_space);
Description	Retrieve the remaining available space of the file system.
Parameter	avail_space: available space
Return Value	Success: **NWY_SUCCESS** Failure: another value
Remarks	The RDA platform needs to know the point of the vfs mount, which is usually defined by the baseline.

3.4 Thread

Perform a thread-related operation. The API definitions can be found in **nwy_osi_api.h**.

3.4.1 nwy_sdk_log_printf

Function	void nwy_sdk_log_printf(nwy_log_level_e lev, char* fmt, ...)
Description	Log printing
Parameter	Lev: log level Fmt: log printing format

Return Value	NA

3.4.2 nwy_thread_create

Function	nwy_error_e nwy_thread_create(nwy_osi_thread_t **hdl, char *task_name, uint8 priority, nwy_task_cb_func cb, void *argv, uint32 event_count, uint32 stack_size, void *stack)
Description	Create a thread
Parameter	hdl: A pointer to a pointer that outputs the created thread handle. task_name: The name of the thread. stack: Specifies the starting address of the stack. Defaults to NULL. stack_size: The size of the stack. priority: The priority level of the thread. cb: The function to be executed by the thread. argv: The parameters to be passed to the callback function cb. event_count: The number of events held by the thread. Each thread has its own event queue.
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.4.3 nwy_thread_get_current

Function	nwy_osi_thread_t *nwy_thread_get_current(void);
Description	Retrieve the handle of the task in which the current function is being executed.
Parameter	NA
Return Value	Success: Returns the handle of the current task. Failure: **NULL**

3.4.4 nwy_thread_get_priority

Function	nwy_error_e nwy_thread_get_priority(nwy_osi_thread_t *hdl, uint8 *priority);
Description	Obtain the priority
Parameter	hdl: thread Priority: priority
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.4.5 nwy_thread_set_priority

Function	nwy_error_e nwy_thread_set_priority(nwy_osi_thread_t *hdl, uint8 new_pri);

Description	Set the thread priority
Parameter	hdl: thread new_pri: new priority
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.4.6 nwy_thread_suspend

Function	nwy_error_e nwy_thread_suspend(nwy_osi_thread_t *hdl);
Description	Suspend a task
Parameter	hdl: thread
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.4.7 nwy_thread_resume

Function	nwy_error_e nwy_thread_resume(nwy_osi_thread_t * hdl);
Description	Wake up a task
Parameter	hdl: thread
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.4.8 nwy_thread_exit

Function	nwy_error_e nwy_thread_exit(nwy_osi_thread_t *hdl);
Description	Exit a thread
Parameter	hdl: thread
Return Value	Success: **NWY_SUCCESS** Failure: another value
Remarks	When a thread needs to be exited, the thread exit function must be executed to release thread resources. • For RDA: If hdl is null, the current thread will exit. If hdl is not null, the specified thread will exit. Before exiting, the task will be suspended. • For ASR (based on ThreadX): The current thread cannot exit within its own context. If hdl is null, the function will return an error parameter. If hdl is not null, the specified thread will exit. Before calling this function to exit a thread, ensure that the corresponding hdl thread is in a suspended state; otherwise, the exit will fail.

3.4.9 nwy_thread_sleep

Function	void nwy_thread_sleep(uint32 ms);
Description	Sleep delay in ms
Parameter	sleep: delay time, unit: ms
Return Value	NA

3.4.10 nwy_thread_usleep

Function	void nwy_thread_usleep(uint32 us);
Description	Sleep delay in us
Parameter	sleep: delay time, unit: μs
Return Value	NA

3.4.11 nwy_thread_event_send

Function	nwy_error_e nwy_thread_event_send(nwy_osi_thread_t *hdl, nwy_event_msg_t *event, int timeout)
Description	Send an event to the thread
Parameter	hdl: thread handle event: event to be sent timeout: Behavior when the message queue is full. 0: no block 1: wait for ever Other values: wait for time ms.
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.4.12 nwy_thread_event_wait

Function	nwy_error_e nwy_thread_event_wait(nwy_osi_thread_t *hdl, nwy_event_msg_t *event, int timeout)
Description	Wait for thread events
Parameter	hdl: thread handle event: Event pointer timeout: Behavior when the message queue is empty. 0: Non-blocking mode. 1: wait for ever until there is an event Others: wait for time ms.
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.4.13 nwy_thread_pendingevt_num_get

Function	nwy_error_e nwy_thread_pendingevt_num_get(nwy_osi_thread_t *hdl, uint32 *pnum)
Description	Retrieve the number of pending events currently in the thread's event queue.
Parameter	hdl: thread handle pnum: Number of pending events
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.4.14 nwy_thread_spaceevt_cnt_get

Function	nwy_error_e nwy_thread_spaceevt_cnt_get(nwy_osi_thread_t *hdl, uint32 *pnum)
Description	Retrieve the available event slots in the thread event queue
Parameter	hdl: thread handle pnum: Number of space events
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.4.15 nwy_thread_info_get

Function	nwy_error_e nwy_thread_info_get(nwy_osi_thread_t *hdl, nwy_thread_info_t *thread_info)
Description	Retrieve information of a specific thread.
Parameter	hdl: thread handle thread_info: Output retrieved thread information.
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.4.16 nwy_thread_list_get

Function	void nwy_thread_list_get(char *thread_list)
Description	Obtain the task list information in the system
Parameter	thread_list: Task list information
Return Value	NA

3.4.17 nwy_thread_runtime_stats_get

Function	nwy_error_e nwy_thread_runtime_stats_get(char *stats_buf)
Description	Get system task runtime statistics
Parameter	stats_buf: Task runtime information

Return Value	Success: **NWY_SUCCESS** Failure: another value

3.5 Mutex Lock

To perform a mutex-related operation. The API definitions can be found in **nwy_osi_api.h**.

3.5.1 nwy_sdk_mutex_create

Function	nwy_error_e nwy_sdk_mutex_create(nwy_osi_mutex_t** mutex)
Description	Initialize a mutex.
Parameter	mutex: A pointer to the mutex being created, represented as a pointer to a pointer.
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.5.2 nwy_sdk_mutex_lock

Function	nwy_error_e nwy_sdk_mutex_lock(nwy_osi_mutex_t *mutex, int time_out);
Description	Lock the specified mutex.
Parameter	mutex: mutual exclusion lock time_out: 0: no block 1: wait for ever Other values: wait for time ms
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.5.3 nwy_sdk_mutex_unlock

Function	nwy_error_e nwy_sdk_mutex_unlock(nwy_osi_mutex_t *mutex);
Description	Unlock the mutex.
Parameter	mutex: mutually exclusive lock
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.5.4 nwy_sdk_mutex_delete

Function	nwy_error_e nwy_sdk_mutex_delete(nwy_osi_mutex_t *mutex);
Description	Delete a mutual exclusive lock
Parameter	mutex: mutually exclusive lock

Return Value	Success: **NWY_SUCCESS** Failure: another value
Remarks	Always release the mutex after usage to avoid resource waste.

3.6 Semaphores

To perform semaphore-related operations. The API definitions can be found in **nwy_osi_api.h**.

3.6.1 nwy_semaphore_create

Function	nwy_error_e nwy_semaphore_create(nwy_osi_semaphore_t **sem,uint32 sem_count)
Description	Initialize a semaphore
Parameter	sem: Semaphore handle, a pointer to a pointer. sem_count: initial count of the semaphore
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.6.2 nwy_semaphore_acquire

Function	nwy_error_e nwy_semaphore_acquire(nwy_osi_semaphore_t *sem, int time_out);
Description	Request a semaphore
Parameter	sem: semaphore timeout: time-out period, unit: ms (0: wait forever)
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.6.3 nwy_semahpore_release

Function	nwy_error_e nwy_semahpore_release(nwy_osi_semaphore_t *sem);
Description	Release a semaphore
Parameter	sem: semaphore
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.6.4 nwy_semahpore_delete

Function	nwy_error_e nwy_semahpore_delete(nwy_osi_semaphore_t *sem);
Description	Delete a semaphore
Parameter	sem: semaphore

Return Value	Success: **NWY_SUCCESS** Failure: another value

3.7 Date and Time

The interface definitions can be found in **nwy_api.h** and the relevant header file is **nwy_common.h**. These APIs are used to read and set the date and time.

3.7.1 nwy_date_set

Function	nwy_error_e nwy_date_set(nwy_time_t *p_time, int timezone)
Description	Set the date and time.
Parameter	p_time: input parameter timezone: Time zone information. The time zone unit is fifteen minutes, equivalent to one-quarter of an hour.
Return Value	Success: **NWY_SUCCESS** Failure: another value
Remarks	1. The valid range for year is 2000–2100. 2.A time zone refers to a region of the Earth divided by longitude, where each region follows a unified standard time. The unit of a time zone is fifteen minutes, equivalent to one-quarter of an hour or 15 minutes. The world is divided into 24 time zones, with each time zone separated by 15 degrees of longitude and differing by 1 hour. This division facilitates people in different regions to follow local time for daily activities and routines.

3.7.2 nwy_date_get

Function	nwy_error_e nwy_date_get(nwy_time_t *p_time, int* timezone)
Description	Obtain the date and time interface
Parameter	julian_time: time (output parameter) timezone: Time zone information. The time zone unit is fifteen minutes, equivalent to one-quarter of an hour.
Return Value	Success: **NWY_SUCCESS** Failure: another value
Remarks	A time zone refers to a region of the Earth divided by longitude, where each region follows a unified standard time. The unit of a time zone is fifteen minutes, equivalent to one-quarter of an hour or 15 minutes. The world is divided into 24 time zones, with each time zone separated by 15 degrees of longitude and differing by 1 hour. This division facilitates people in different regions to follow local time for daily activities and routines.

3.7.3 nwy_uptime_get

Function	int64 nwy_uptime_get(void)
Description	Query how long the system has been running since the last startup.
Parameter	N/A.
Return Value	time in units of ms

3.7.4 nwy_sys_timestamp_set

Function	nwy_error_e nwy_sys_timestamp_set(nwy_timeval_t *time);
Description	Set the system timestamp.
Parameter	* time: The timestamp, representing the number of seconds elapsed since January 1, 1970 (UTC).
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.7.5 nwy_sys_timestamp_get

Function	nwy_error_e nwy_sys_timestamp_set(nwy_timeval_t *time);
Description	Retrieve the system timestamp.
Parameter	* time: The timestamp, representing the number of seconds elapsed since January 1, 1970 (UTC).
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.7.6 nwy_sys_timezone_set

Function	nwy_error_e nwy_sys_timezone_set(int tz);
Description	Set up system time zone
Parameter	tz: Time zone information. The time zone unit is fifteen minutes, equivalent to one-quarter of an hour.
Return Value	Success: **NWY_SUCCESS** Failure: another value
Description	1. The interfaces nwy_date_get and nwy_date_get also support querying and setting time zone information 2. A time zone refers to a region of the Earth divided by longitude, where each region follows a unified standard time. The unit of a time zone is fifteen minutes, equivalent to one-quarter of an hour or 15 minutes. The world is divided into 24 time zones, with each time zone separated by 15 degrees of longitude and differing by 1 hour.

	This division facilitates people in different regions to follow local time for daily activities and routines. 3. For example: When calling nwy_sys_timezone_set(32); , it sets the system time zone to 32 fifteen-minute units. Each unit equals 15 minutes, so 32 units correspond to: 32 * 15 = 480 minutes = 8 hours. Thus, this call sets the system time zone to UTC+8, 8 hours ahead of UTC time.

3.8 Timer

Perform timer-related operations. The API definitions can be found in **nwy_osi_api.h**.

3.8.1 nwy_sdk_timer_create

Function	nwy_error_e nwy_sdk_timer_create(nwy_osi_timer_t **timer, nwy_timer_para_t *para)
Description	Create a timer
Parameter	timer: A pointer to a pointer, pointing to the pointer of the created timer. A pointer to a pointer, pointing to the pointer of the created timer. para: Parameters for creating timer, see nwy_timer_para_t structure for details
Return Value	Success: Timer pointer Failure: **NULL**

3.8.2 nwy_sdk_timer_destory

Function	nwy_error_e nwy_sdk_timer_destory(nwy_osi_timer_t *timer)
Description	Delete a timer
Parameter	timer: timer pointer
Return Value	N/A.
Remarks	Ensure to call this function after using the timer to prevent memory leaks.

3.8.3 nwy_sdk_timer_start

Function	nwy_error_e nwy_sdk_timer_start(nwy_osi_timer_t *timer, nwy_timer_para_t *para)
Description	Starts a timer to begin counting.
Parameter	timer: Pointer to the timer being started. para: Parameters for creating the timer, defined in the nwy_timer_para_t structure.
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.8.4 nwy_sdk_timer_stop

Function	nwy_error_e nwy_sdk_timer_stop(nwy_osi_timer_t *timer)
Description	Stops a timer
Parameter	timer
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.9 Log

This interface function is defined in nwy_log.h and is used for debugging log output.

```
/* LOG LEVERL DEFINITION */
#define LOG_EMERG       0          /* systemis unusable */
#define LOG_ALERT       1          /* actionmust bet taken immediately */
#define LOG_CRIT        2          /* critical conditions */
#define LOG_ERR         3          /* error conditions */
#define LOG_WARNING     4          /* warning conditions */
#define LOG_NOTICE      5          /* normalbut significant */
#define LOG_INFO        6          /* informational */
#define LOG_DEBUG       7          /* debug-level messages */

#define NWY_LOG_OUTPUT_LEVEL       7

#define NWY_LOG_PRINTF(level, tag, fmt, ...)
    do
    {
        if (NWY_LOG_OUTPUT_LEVEL >= level)
            osiTracePrintf("nwy open ", fmt, ##__VA_ARGS__);
    } while (0)

#define NWY_LOGV(level, fmt, args...)  NWY_LOG_PRINTF(level, fmt, ###args)
#define NWY_LOGI(fmt, args...)  NWY_LOG_PRINTF(LOG_INFO, fmt, ###args)
#define NWY_LOGE(fmt, args...)  NWY_LOG_PRINTF(LOG_ERR, fmt, ###args)
#define NWY_LOGD(fmt, args...)  NWY_LOG_PRINTF(LOG_DEBUG, fmt, ###args) nwy_open_sdk_log
```

3.10 Socket

3.10.1 nwy_socket_errno

Function	int nwy_socket_errno(void);
Description	Get the socket error code
Parameter	NA
Return Value	The corresponding error code, see **errno.h**

3.10.2 nwy_socket_open

Function	int nwy_socket_open(int domain, int type, int protocol, int cid);
Description	Create a socket
Parameter	domain: Protocol family, commonly used families include: AF_INET (Ipv4)/ AF_INET6 (IPv6) type: Socket type. protocol: Communication protocol used, such as IPPROTO_TCP or IPPROTO_UDP. cid: Link CID, ranging from 1 to 7. For the default link, use NWY_DEFAULT_NETIF_ID.
Return Value	A socket ID.

3.10.3 nwy_socket_send

Function	int nwy_socket_send(int socket, const void *data, size_t size, int flags);
Description	Send data over a designated socket
Parameter	socket: socket id size: size of the data sent flags: Unused, set to 0.
Return Value	The actual length of data sent.

3.10.4 nwy_socket_recv

Function	int nwy_socket_recv(int socket, void *data, size_t len, int flags);
Description	Receive data via a designated socket.
Parameter	socket: socket id data: receive data buffer. len: length of the buffer. flags: unused, set to 0.
Return Value	The actual length of data received.

3.10.5 nwy_socket_sendto

Function	int nwy_socket_sendto(int socket, const void *data, size_t size, int flags,const struct sockaddr *to, socklen_t tolen);
Description	Send data to a specified address via a designated socket.
Parameter	socket: socket ID size: size of the data sent flags: Unused, set to 0. to: destination

	tolen: length of the destination address
Return Value	The actual length of data sent.

3.10.6 nwy_socket_recvfrom

Function	int nwy_socket_recvfrom(int socket, void *data, size_t len, int flags,struct sockaddr *from, socklen_t *fromlen);
Description	Receive data from a specified address via a socket.
Parameter	socket: socket id data: buffer to store received data. len: length of the buffer. flags: unused, set to 0. from: specified address. fromlen: length of the specified address.
Return Value	Returns the actual length of the received data.

nwy_socket_setsockopt

Function	nwy_error_e nwy_socket_setsockopt(int socket, int level, int optname, const void * optval, socklen_t optlen);
Description	Set the socket option
Parameter	socket: socket descriptor level: the layer of protocol to which this option will be applied, e.g. SOL_SOCKET, IPPROTO_IP, IPPROTO_TCP optname: option name to be accessed optval: points to the buffer containing new options optlen: option length
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.7 nwy_socket_getsockopt

Function	nwy_error_e nwy_sdk_socket_getsockopt(int socket, int level, int optname, void * optval, socklen_t * optlen);
Description	Return the socket option value
Parameter	socket: socket descriptor level: the layer of protocol to which this option will be applied optname: option name to be accessed optval: points to the buffer returning option values optlen: maximum length of the option value

Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.8 nwy_socket_gethost_by_name

Function	char* nwy_socket_gethost_by_name(const char *name, int *isipv6,int cid);
Description	Obtain a host address according to a domain name
Parameter	name: domain name isipv6: specifies whether the return value is an IPv6 address; 1: ipv6, 0: ipv4 cid: Link CID, ranging from 1 to 7. For the default link, use NWY_DEFAULT_NETIF_ID.
Return Value	Success: address of the host IP Failure: NULL

3.10.9 nwy_socket_close

Function	nwy_error_e nwy_socket_close(int socket);
Description	Close a socket
Parameter	socket: socket descriptor
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.10 nwy_socket_connect

Function	nwy_error_e nwy_socket_connect(int socket, const struct sockaddr *name, socklen_t namelen);
Description	Connect a socket to a server.
Parameter	socket: socket descriptor name: server socket address. namelen: length of the server socket length
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.11 nwy_socket_bind

Function	nwy_error_e nwy_sdk_socket_bind(int socket, struct sockaddr *addr, socklen_t *addrlen)
Description	Bind a socket connection
Parameter	socket: socket descriptor addr: socket information addrlen: length
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.12 nwy_socket_listen

Function	nwy_error_e nwy_socket_listen(int socket, int backlog)
Description	Create a socket listener
Parameter	socket: socket descriptor backlog: maximum number of clients that can be listened
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.13 nwy_socket_fcntl

Function	int nwy_socket_fcntl(int socket, int cmd, int value)
Description	Perform various control operations on an open file descriptor to modify its attributes.
Parameter	socket: socket descriptor cmd: command to set, e.g., F_GETFL, F_SETFL. value: determined by the second parameter. For get operations, it can be set to 0. For set operations, it specifies the value to set, such as O_NONBLOCK.
Return Value	Success: a value related to the specific command. Failure: -1

3.10.14 nwy_socket_accept

Function	nwy_error_e nwy_socket_accept (int socket, struct sockaddr *addr, socklen_t *addrlen)
Description	Receive a socket connection
Parameter	socket: socket descriptor addr: socket information addrlen: length
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.15 nwy_socket_select

Function	int nwy_socket_select(int maxfdp1, fd_set *readset, fd_set *writeset, fd_set *exceptset, struct timeval *timeout)
Description	Select a socket listener
Parameter	maxfdp1: range of all the file descriptors, that is, the maximum value of the file descriptor plus 1. readset: to monitor the read status of file descriptors; a value greater than 0 indicates there are readable files writeset: to monitor the write status of file descriptors; a value greater than 0 indicates there are writable files.

	exceptset: to monitor file abnormalities timeout: timeout period of the select function • NULL: the select function blocks • 0s 0ms: the select function is set to a pure non-blocking function. Regardless of whether the file descriptor changes, a value is returned immediately to continue performing its function. If the file does not change, 0 is returned. If the file changes, a positive value is returned. • A value greater than 0: timeout period If an event occurs within the period, a value greater than 0 is returned. A value must be returned when the timeout period is over.
Return Value	• The select function encounters abnormalities: a value lower than 0 is returned. • Some files can be read or written: a value greater than 0 is returned. • The function times out and no files can be read or written or the files are wrong: 0 is returned.
Remarks	After a message is received by using the SELECT mechanism, send this message to another thread (e.g. thread A) through an Event, and the message will be processed in the thread. Otherwise, the message will fail to be processed since the SELECT function is blocked, when it is used, which affects the event reception.

3.10.16 nwy_socket_shutdown

Function	nwy_error_e nwy_socket_shutdown(int socket, int how)
Description	Closes one end of a full-duplex socket connection.
Parameter	• socket: socket descriptor • how: SHUT_RD ,SHUT_WR, SHUT_RDWR
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.17 nwy_socket_get_tcp_state

Function	nwy_tcp_state_e nwy_socket_get_tcp_state(int socket)
Description	Obtain the TCP connection status.
Parameter	socket: socket descriptor
Return Value	Status Code

3.10.18 nwy_socket_lport_bind

Function	nwy_error_e nwy_socket_lport_bind(int socket, uint16_t lport);
Description	Bind a socket to a specific port.
Parameter	• socket: socket ID • lport: port number

Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.19 nwy_socket_set_nonblock

Function	nwy_error_e nwy_socket_set_nonblock(int socket);
Description	Set the socket to non-blocking mode.
Parameter	• socket: socket ID
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.20 nwy_socket_set_reuseaddr

Function	nwy_error_e nwy_socket_set_reuseaddr(int socket);
Description	Set a socket to reuse an address.
Parameter	socket: socket ID
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.21 nwy_socket_set_nodelay

Function	nwy_error_e nwy_socket_set_nodelay(int socket);
Description	Set TCP_NODELAY option, to control whether to enable or disable the Nagle algorithm for TCP sockets.
Parameter	socket: socket ID
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.22 nwy_socket_set_keepalive

Function	nwy_error_e nwy_socket_set_keepalive(int socket ,uint32_t mode ,uint32_t idle ,uint32_t interval);
Description	Set TCP keepalive
Parameter	• socket: socket ID • mode: 1: enable, 0: disable • idle: idle time interval before the first keepalive packet is sent. • nterval: interval between successive keepalive packets
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.23 nwy_socket_set_linger

Function	nwy_error_e nwy_socket_set_linger(int socket);
Description	Set socket linger
Parameter	socket: socket ID
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.24 nwy_socket_get_ack

Function	int nwy_socket_get_ack(int socket);
Description	Retrieve the number of ACKs received by the socket.
Parameter	socket: socket ID
Return Value	The total number of ACKs received, representing the amount of data successfully sent.

3.10.25 nwy_socket_get_sent

Function	int nwy_socket_get_sent(int socket);
Description	Retrieve the total amount of data sent via the socket.
Parameter	socket: socket ID
Return Value	The total data sent via the socket.

3.10.26 nwy_socket_inet_ntop

Function	nwy_error_e nwy_socket_inet_ntop(int family, void *src, char *dst, int size);
Description	Convert an IPv4 or IPv6 Internet network address to its standard textual representation.
Parameter	• The protocol type, AF_INET (IPv4) or AF_INET6 (IPv6). • src: The source network address. • dst: The destination buffer for the converted address string. • size: The length of the source network address.
Return Value	Success: **NWY_SUCCESS** Failure: another value

3.10.27 nwy_socket_inet_pton

Function	nwy_error_e nwy_socket_inet_pton(int family, char *src, void *dst);
Description	Convert a standard textual IPv4 or IPv6 Internet address to binary form.
Parameter	• The protocol type, AF_INET (IPv4) or AF_INET6 (IPv6). • src: the IP address string. • dst: the destination buffer for the binary address.

Return Value	Success: **NWY_SUCCESS** Failure: another value

3.11 Virtual AT

3.11.1 nwy_virt_at_parameter_init

Function	nwy_error_e nwy_virt_at_parameter_init();
Description	Initialize the virtual AT
Parameter	NA
Return Value	The corresponding error code, see **errno.h**

3.11.2 nwy_virt_at_cmd_send

Function	nwy_error_e nwy_virt_at_cmd_send(nwy_at_info_t *pInfo, char *resp, int resp_len, int timeout);
Description	Send virtual AT
Parameter	pInfo: virtual AT information resp_ len: buffer length of response code timeout: default value not exceeding 30s. resp: response code information
Return Value	The corresponding error code, see **errno.h**

3.11.3 nwy_virt_at_unsolicited_cb_reg

Function	nwy_error_e nwy_virt_at_unsolicited_cb_reg(char *at_prefix, void *p_func);
Description	Register an unsolicited result code (URC)
Parameter	at_prefix: the prefix for the URC. p_func: the callback function to handle the URC
Return Value	The corresponding error code, see **errno.h**

3.11.4 nwy_virt_at_get_original_rsp_cb

Function	nwy_error_enwy_virt_at_get_original_rsp_cb(nwy_get_original_at_rsp cb);
Description	Register a callback function to retrieve the original AT response.
Parameter	Cb: Callback function for processing the original AT response.
Return Value	The corresponding error code, see **errno.h**

3.11.5 nwy_virt_at_forward_cb

Function	nwy_error_e nwy_virt_at_forward_cb(int index,const char* name, nwy_at_forward_process_cb cb);
Description	Register a custom AT command.
Parameter	index: serial number name: AT command name cb: Callback function for processing the custom AT command.
Return Value	The corresponding error code, see **errno.h**

3.11.6 nwy_virt_at_forward_send

Function	nwy_error_e nwy_virt_at_forward_send(void* handle, char* buf, int len);
Description	Send a custom AT command.
Parameter	handle: UART, USB, or virtual AT handle. buf: AT command parameters.
Return Value	The corresponding error code, see **errno.h**